



Stealing Part of a Production Language Model

Carlini et al. — ICML 2024 — arXiv:2403.06634

Presented by Sri Bhargava Bhamidi · X-Lab research cohort

A number the industry will not print

MODEL WIDTH IS WITHHELD ON PURPOSE.

GPT-4 technical report

Declines to disclose **architecture**, **model size**, hardware or training method.

PaLM-2 technical report

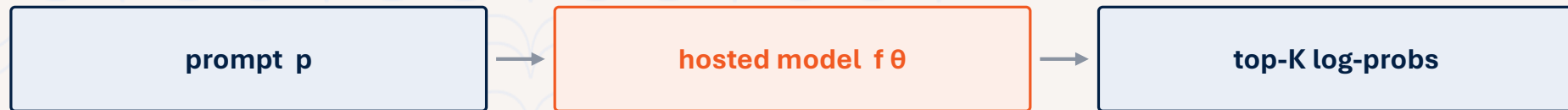
States that **model size** and **architecture** details are withheld from publication.

- Width is not a insignificant statistic. it **correlates with total parameter count** and exposes relative sizes across a model family.
- Also, if the Width is known, then the **entire exact sub dimension of embedding projection layer** could be constructed.

So: how much can an attacker learn about a production model **using only its API?**

All you are allowed to touch (Challenges)

THE THREAT MODEL: A PERFECT BLACK BOX.



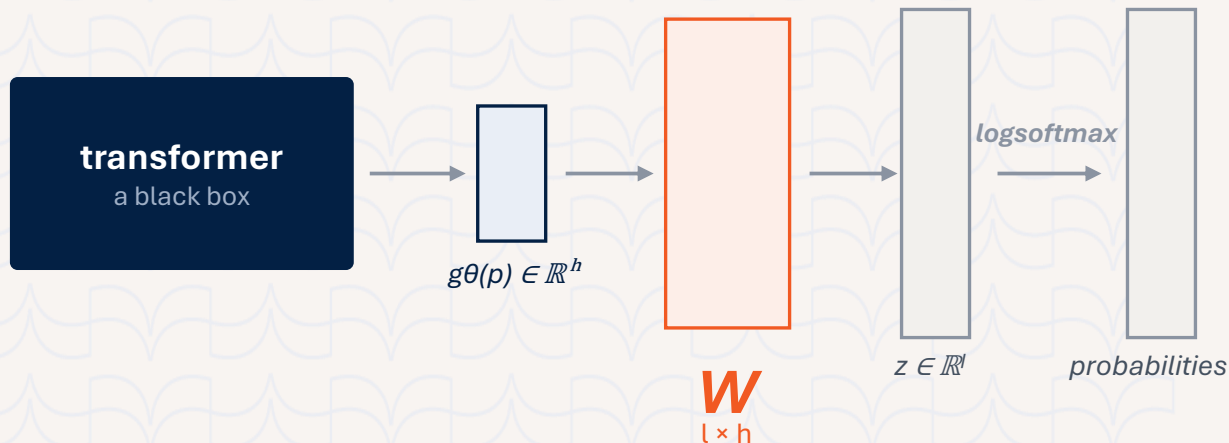
- **No weights, no gradients, no activations.**
- the API leaks nothing beyond the prompt and the response itself
- You pay per token, so every attack has a dollar price. Cost is a first-class constraint in this paper.

The attacker knows nothing about the parameters. Only the API response.

What a language model does at the very end

ONE MATRIX MULTIPLY, THEN A SOFTMAX.

$$f_{\theta}(p) = \text{log_softmax}(\mathbf{W} \cdot \mathbf{g}_{\theta}(p))$$



- Everything upstream is irrelevant to us.
- For this paper the transformer is just a machine that emits **one vector** of length h .

Notation used on every slide that follows

ONE TABLE. NOTHING GETS RENAMED LATER.

p one prompt (a distinct input)

$g\theta(p)$ hidden state the model emits, in $\mathbb{R}^h$

h hidden dimension — the model's width

l vocabulary size = number of logits

W embedding projection matrix, $l \times h$

z logit vector for one prompt, $z = W \cdot g\theta(p)$

n number of prompts = columns of Q

Q collected logit vectors, $l \times n$

H hidden states stacked, $h \times n$

U, Σ, V the SVD factors of Q

λ_i singular values — the diagonal of Σ

G unknown invertible $h \times h$ basis change

$\tilde{W}$ what we recover: $\tilde{W} = W \cdot G$

b, B logit-bias vector / a large bias value

Matrices are capitals — mind h (the width, a number) versus H (a matrix). Singular values are λ

PART 0

WHY A MATRIX CALLED Q?

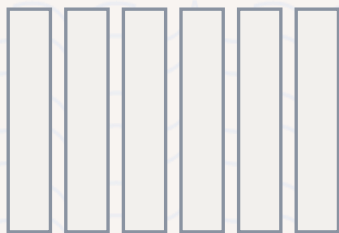
Before any attack: what Q is, and why every logit vector a model can produce is trapped inside a vanishingly thin slice of a huge space.

How is the Q Matrix made?

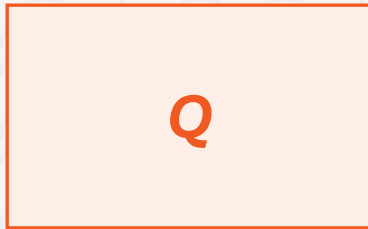
COLLECT MANY AND YOU HAVE THE MATRIX Q.

$$W \cdot g\theta(p) = z$$

one prompt $\rightarrow$ one logit vector



$z_1 \dots z_n$ (each of length l)



$l \times n$

$$Q = [z_1 \ z_2 \ \dots \ z_n]$$

n prompts $\rightarrow$ the matrix Q

- Each column is one prompt's logits.
- The prompts can be **anything** — random text works fine.
- Q is the only object we can actually build from outside.

$$Q = W \cdot H \quad \text{where } H \text{ stacks the hidden states we never see}$$

The claim

STATED NOW — PROVED ON THE NEXT TWO SLIDES.

Every column of Q lies in the same **h -dimensional subspace** of “ l ”
Dimension.

- This is true no matter which prompts you pick, or how many.
- Because $h \ll l$, that subspace is **vanishingly thin**.

Proof — read it column by column

MATRIX TIMES MATRIX, ONE OUTPUT COLUMN AT A TIME.

j-th column of $\mathbf{Q}$ = $\mathbf{W} \cdot (\text{j-th column of } \mathbf{H})$

$$\mathbf{W} \cdot \mathbf{g}\theta(\mathbf{p}_j) = g_1 \mathbf{w}_1 + g_2 \mathbf{w}_2 + \cdots + g_h \mathbf{w}_h$$

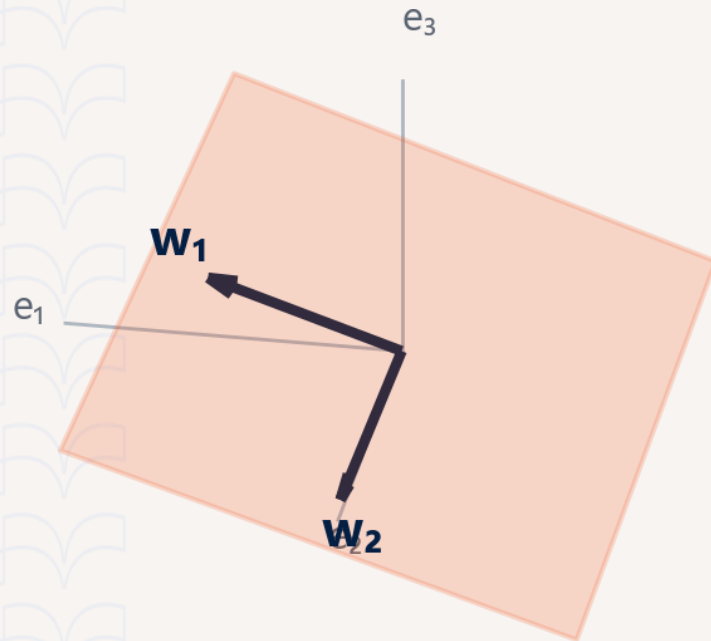
- $\mathbf{w}_1 \dots \mathbf{w}_h$ are the **h columns of $\mathbf{W}$** . The numbers $g_1 \dots g_h$ are just the entries of that prompt's hidden state.
- So every output column is a **linear combination of the same h vectors**. the coefficients change from prompt to prompt, the vectors never do.

every column of $\mathbf{Q} \in \text{span}\{\mathbf{w}_1, \dots, \mathbf{w}_h\} \Rightarrow$ at most **h** independent directions, ever

The same statement, in three dimensions

TOY MODEL: 3 TOKENS, WIDTH 2. SO W IS 3×2 .

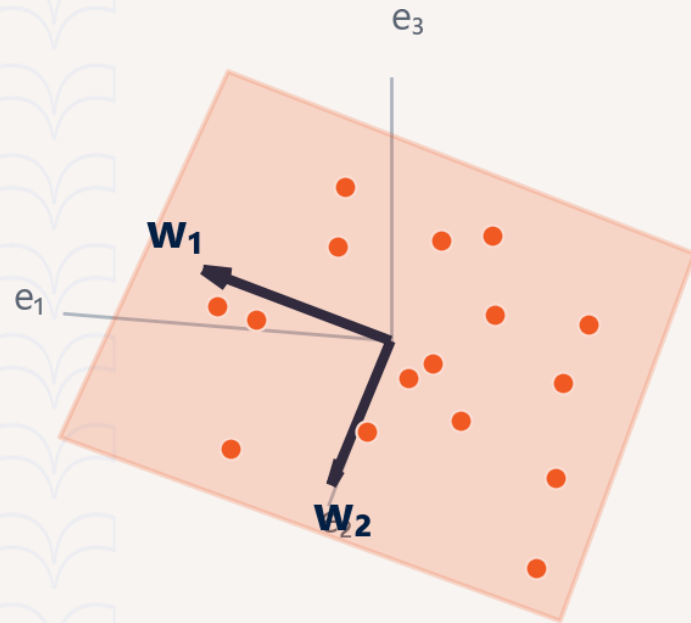
- W has exactly **two** columns: w_1 and w_2 .
- Any product $W \cdot h$ is $c_1 w_1 + c_2 w_2$ for some pair of numbers.
- The set of all such combinations is a **plane through the origin** — a 2-dimensional subspace sitting inside 3-dimensional space.
- **It does not matter what h is. You cannot leave the plane.**



Every output lands on that plane

NEVER OFF IT — HOWEVER MANY PROMPTS YOU TRY.

- Each dot is one prompt's logit vector.
- The third dot was already a combination of the first two — and every dot after it adds **nothing new**.



$3 \times 2 \rightarrow 50,257 \times 768$ (GPT-2)

same picture, real scale: a 768-dimensional sliver inside 50,257 dimensions

The rank of Q is the model's width

LEMMA 4.1 — THE HINGE THE PAPER TURNS ON.

$$\begin{matrix} \boxed{Q} \\ l \times n \end{matrix} = \begin{matrix} \boxed{W} \\ l \times h \end{matrix} \cdot \begin{matrix} \boxed{H} \\ h \times n \end{matrix}$$

$$\text{rank}(\mathbf{Q}) \leq \min(\text{rank } \mathbf{W}, \text{rank } \mathbf{H}) \leq h$$

- With diverse prompts both factors have full rank h , so the inequality becomes an **equality**: $\text{rank}(\mathbf{Q}) = h$.
- The paper notes the failure case would need every hidden state to collapse into a smaller subspace — which does not happen in practice.

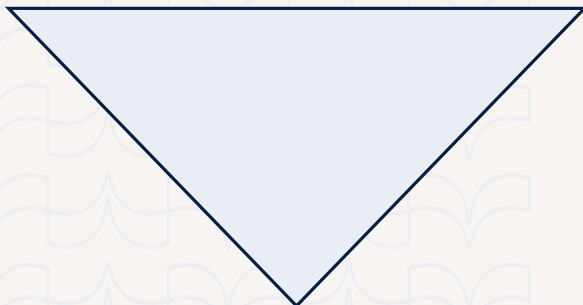
Measure the rank of what you collected → **you have read the model's width**

So the whole attack reduces to one question

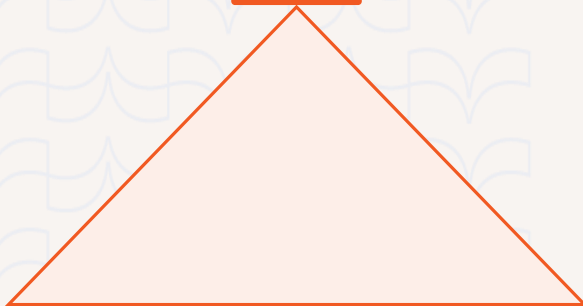
CAN WE BUILD Q ? AND WHAT FALLS OUT ONCE WE HAVE IT?

- **Query the API.**
- Reconstruct one full logit vector per prompt — the hard engineering half.

PART 1 — getting Q



Q



PART 2 — what Q gives you

- **One SVD.**
- Finding 1: the hidden dimension h .
- Finding 2: the projection matrix $\tilde{W}$.

PART 1

GETTING Q

The top of the hourglass. Q is built out of logit vectors, and the API will not give you a logit vector.

What we need versus what we are given

THE GAP THIS ENTIRE SECTION EXISTS TO CLOSE.

WE NEED

z to form **Q**. the complete logit vector:
all $l \approx 100,000$ entries, for each of **n**
prompts.

WE ARE GIVEN

The top 5 log-probabilities, plus a
logit_bias parameter.

Two separate problems: the numbers are the **wrong quantity**, and we only see **five of them**.

Obstacle 1 — log-probs are not logits

A NORMALISER SITS ON TOP OF EVERY READING.

$$y_i = z_i - \log \sum_j \exp(z_j)$$

what the API returns

the logit we want

the term that ruins everything

Obstacle 2 — you see only five

THE API RETURNS THE TOP K ENTRIES, K = 5.



the full logit vector — only the five tallest are reported

The one knob: `logit_bias`

ADD A CHOSEN VALUE TO CHOSEN TOKENS, PRE-SOFTMAX.

$$\mathcal{O}(p, \mathbf{b}) = \text{TopK}(\text{logsoftmax}(\mathbf{W} \cdot \mathbf{g}\theta(p) + \mathbf{b}))$$

Designed for

Controlled and constrained generation; a feature steering a model without fine-tuning it.

OpenAI's limits

At most 300 tokens per request; each bias in the range $[-100, +100]$.

What it really is

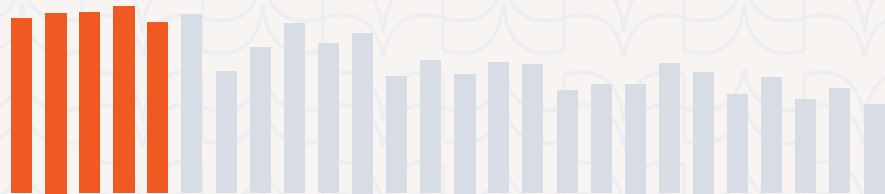
A write primitive into the model's logits. This is the entire attack surface.

Two innocuous convenience features — top-K log-probs and a bias knob — **compose into a leak.**

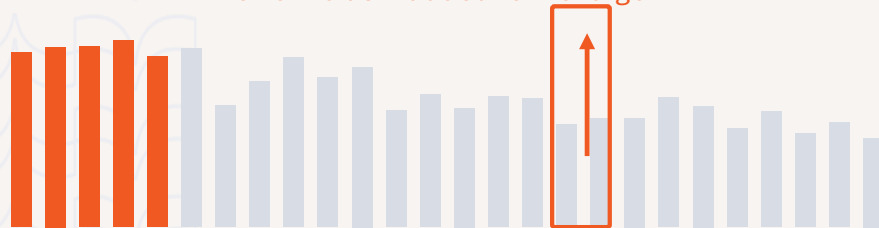
Trick 1 — promotion

PUSH A TOKEN YOU CARE ABOUT INTO THE VISIBLE TOP-5.

before: target token is invisible



after: bias B added to the target



- Adding a large bias B to token i raises its logit by exactly B — enough to force it into the reported top-5.
- Now you can read a number for it. But that number is still a **log-prob**, still carrying the normaliser, and now carrying B as well.
- **Promotion solves obstacle 2. It does not solve obstacle 1.**

Trick 2 — cancel with a reference token

SUBTRACT TWO READINGS; THE UNKNOWN DIES.

one query: promote token **i** with bias **B**, keep the top token **R** in view. Write **D** for the shared softmax denominator.

$$y_R^B = z_R - \log D$$

$$y_i^B = z_i + B - \log D$$

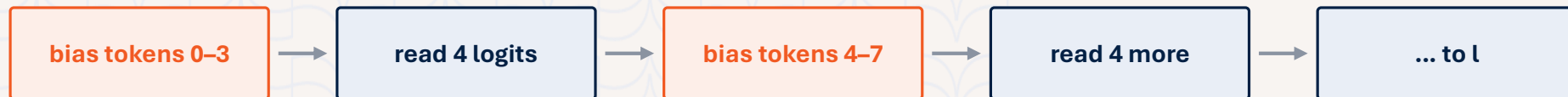
same query, so D is the very same number in both — subtract and it is gone

$$z_R - z_i = y_R^B - y_i^B + B$$

- The unknown denominator cancels; the bias you chose is known. What remains is a **pure difference of true logits**.
- **No approximation anywhere, an exact identity.**

Sweeping the whole vocabulary

FOUR FRESH LOGITS PER QUERY, PLUS THE REFERENCE.



What you recover: logits up to one constant

AND THAT IS ALL ANYONE COULD EVER RECOVER.

$$\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c \cdot \mathbf{1}) \quad \text{for every constant } c$$

- Shifting every logit by the same amount leaves the output distribution **exactly unchanged**.
- So the absolute logits are **not a fact about the model** that any API could ever expose.
- We set $z_R = 0$ and express everything relative to the reference token.
- This costs us nothing: Q's rank, its column space, and h are all **unaffected by a constant shift**.

One complete logit vector

ALL OF THAT MACHINERY BUYS A SINGLE COLUMN OF Q.



- At 0.25 queries per logit and $l \approx 100,000$, one column costs on the order of 25,000 queries.
- Then you repeat the whole thing for a **different prompt** to get the next column.
- This is why the paper is obsessed with queries-per-logit: it is the difference between a \$20 attack and an impossible one.

We can now build Q one column at a time. **How many columns do we need?**

How many columns do we need?

THE QUESTION THAT TURNS THE HOURGLASS OVER.

We need $n > h$ columns.

But h is precisely the number we are trying to discover.

- Too few columns and the rank you measure is just n (basically means n is still less than “ h ”). it tells you nothing about the model.
- Too many and you have burned money for no extra information.
- **And you cannot look up h anywhere: that is the secret.**

The resolution: **Q answers this question about itself.**

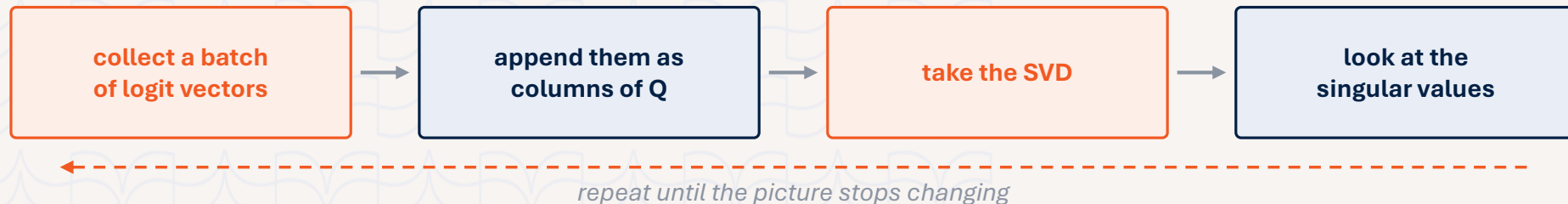
PART 2

WHAT Q GIVES YOU

The bottom of the hourglass. A single decomposition of Q yields both of the paper's findings.

Do not guess n — grow it

LET THE SVD TELL YOU WHEN TO STOP.



- We never need to know n in advance. We watch a **signal** that tells us when we have enough.
- That signal is the shape of the singular value spectrum. and it is unmistakable.

The SVD, in one slide

THREE FACTORS. THE MIDDLE ONE IS ALL WE NEED.

$$Q = U \Sigma V^T$$

$Q \ l \times n$

$U \ l \times n$

$\Sigma \ n \times n$

$V^T \ n \times n$

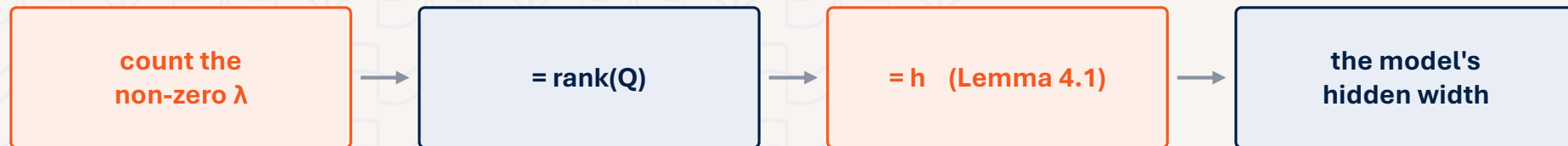
compact SVD, with $n \leq l$ as in the attack — the full SVD would pad U out to $l \times l$

- Σ is diagonal. Its entries $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$ are the **singular values** — how much genuine signal lives along each independent direction.
- The number of non-zero λ is exactly **rank(Q)**.
- U 's first columns are an **orthonormal basis for the column space of Q** — remember that; Finding 2 depends on it.

We write singular values as λ . This paper uses ε for something else entirely — a binary-search tolerance.

Why that is exactly what we want

CHAINING PART 0 TO THE DECOMPOSITION.

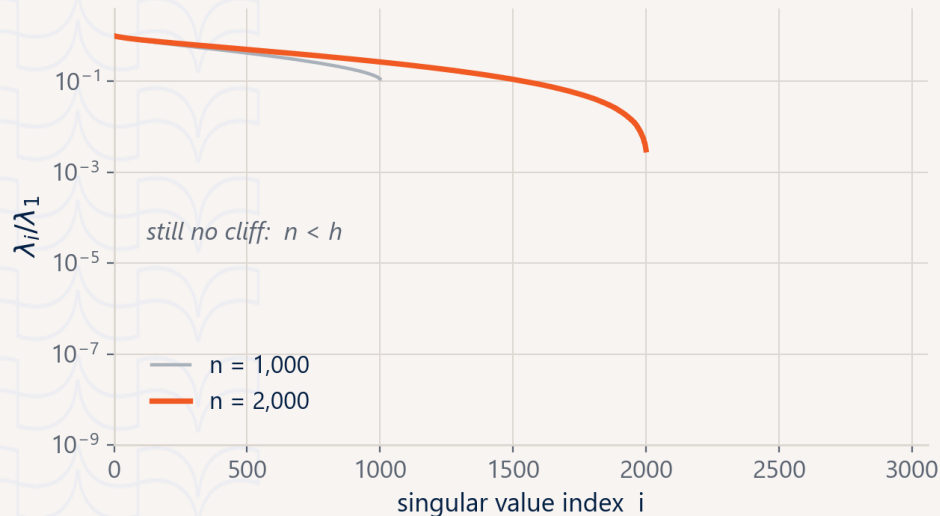


- Part 0 proved $\text{rank}(Q) = h$. The SVD measures $\text{rank}(Q)$. Composing the two reads h straight off the API.
- Nothing here depends on knowing the architecture, the tokenizer, or the training data.
- **One catch: in floating point, 'non-zero' needs defining. That is the next three slides.**

$n = 1,000$ and $n = 2,000$ — nothing yet

EVERY QUERY IS STILL ADDING A BRAND-NEW DIRECTION.

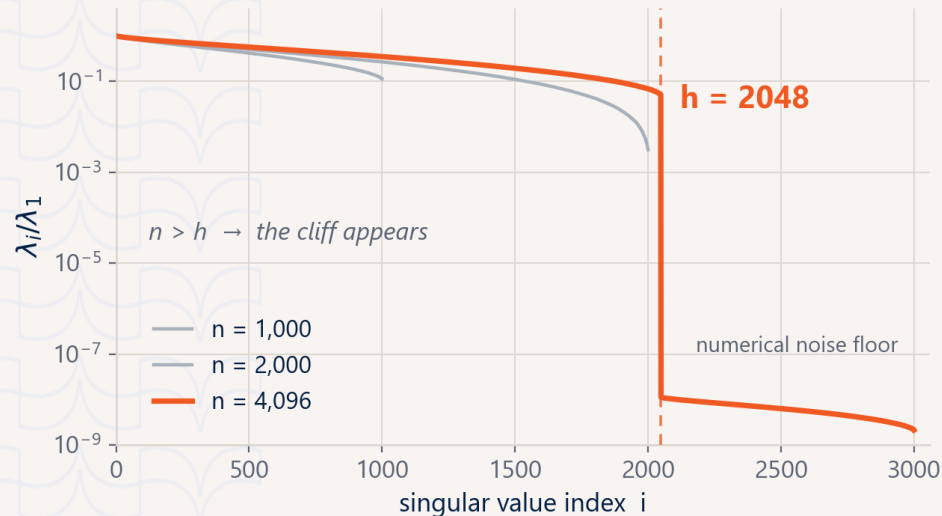
- Both runs are **below** the true $h = 2,048$.
- With $n < h$, Q has full rank n — so you get n large singular values and no cliff at all.
- You cannot yet distinguish 'the model is 2,048 wide' from 'I have not queried enough'.
- At this stage the attack has learned nothing.**



$n = 4,096$ — the cliff appears

LARGE SINGULAR VALUES STOP ARRIVING AT EXACTLY 2,048.

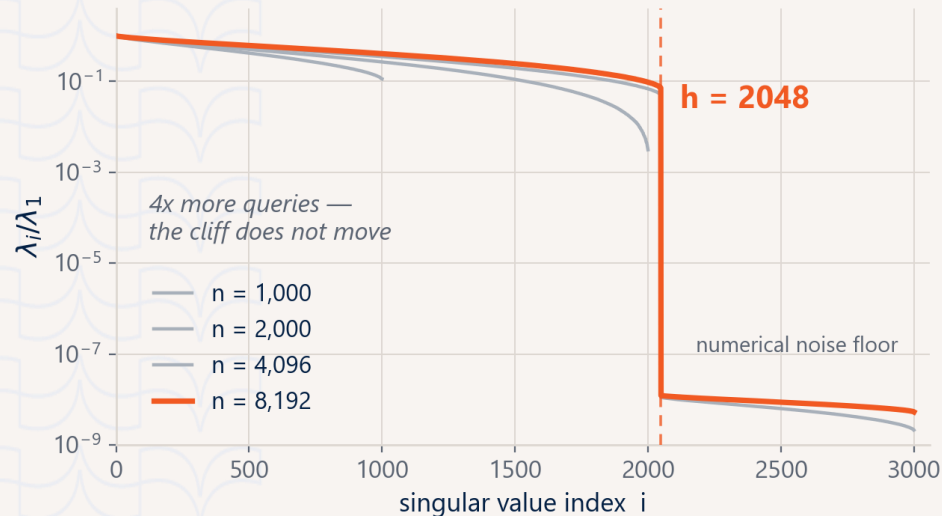
- Now $n > h$. Past index 2,048 the values collapse by **eight orders of magnitude**.
- Above the cliff: real structure. Below it: floating-point residue.
- **The position of the drop is the number we came for.**



$n = 8,192$ — and it does not move

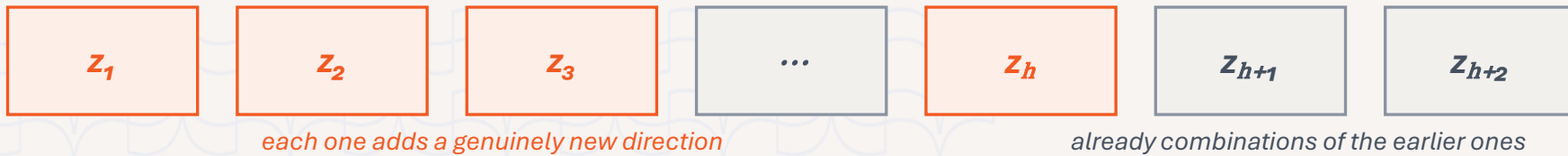
FOUR TIMES THE QUERIES. SAME 2,048. THAT IS THE PROOF.

- If 2,048 were an artefact of how much we queried, it would keep climbing with n . **It does not.**
- The count is pinned — this is the model's own ceiling, not ours.
- **This is the honest answer to 'how do you know when to stop': you stop when more queries stop changing the answer.**



Why 'near zero' means stop

NEW PROMPTS STOP PRODUCING NEW DIRECTIONS.



- The subspace has only h directions in it. Once you have h independent logit vectors, you have **exhausted it**.
- Vector number $h+1$ is forced to be a linear combination of what you already hold — so it contributes a singular value of **zero** (in exact arithmetic).
- Back to the toy plane: once you have two independent points on it, every further point is a mix of those two. The plane is only two-dimensional.

Finding 1 (THE MAIN) — the hidden dimension

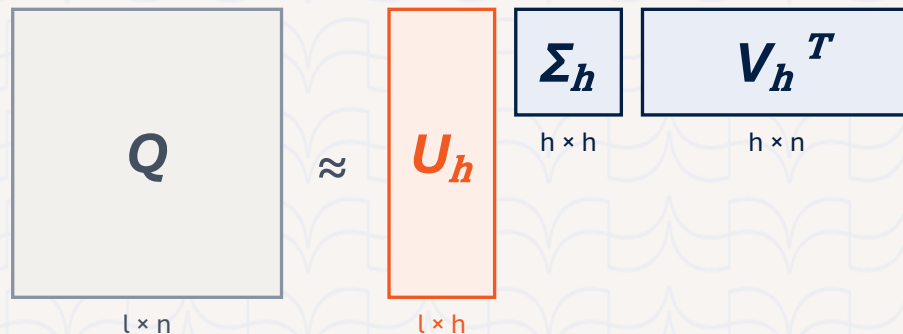
VALIDATED ON OPEN MODELS WHERE THE TRUTH IS KNOWN.

Model	True h	Recovered	$\tilde{W}$ error (RMS)
GPT-2 Small (fp32)	768	757 ± 1	$4 \cdot 10^{-4}$
GPT-2 XL (fp32)	1600	1599 ± 1	$6 \cdot 10^{-4}$
Pythia-1.4B (fp16)	2048	2047 ± 1	$3 \cdot 10^{-5}$
Pythia-6.9B (fp16)	4096	4096 ± 1	$4 \cdot 10^{-5}$
LLaMA 7B (fp16)	4096	4096 ± 2	$8 \cdot 10^{-5}$
LLaMA 65B (fp16)	8192	8192 ± 2	$5 \cdot 10^{-5}$

- **Exact, or off by one, in five of six cases.**
- The exception is honest rather than broken: GPT-2 Small really does use only **757 effective directions** at bf16 precision. In float64 all 768 reappear, but the last dozen are minuscule. The attack recovered the **effective** rank.

Now truncate

KEEP THE REAL DIRECTIONS; DISCARD THE NOISE.



- What we throw away sits $\sim 10^{-8}$ below the top singular value. it is **numerical residue, not signal**.
- This is the standard compact SVD, not a liberty.
- Eckart-Young**: the truncated SVD is provably the best possible rank- h approximation of Q . Nothing better exists.

Truncating is not an approximation. it is the **optimal** rank- h summary of everything we collected.

From $U \cdot \Sigma$ to the weight matrix

TWO LINES OF ALGEBRA, AND W FALLS OUT.

$$Q = W \cdot H$$

from Part 0

$$Q = U \Sigma V^T$$

the SVD we just computed

$$\Rightarrow W \cdot H \cdot V = U \cdot \Sigma$$

V is orthogonal, so $V^T \cdot V = I$

$$\Rightarrow G := H \cdot V \quad (h \times h)$$

a square, invertible mixing matrix

$$\tilde{W} := U \cdot \Sigma = W \cdot G$$

Lemma 4.2(i) — we recover W , multiplied by an unknown $h \times h$ matrix

Finding 2 — why $\tilde{W}$ is still the prize

SAME SHAPE, SAME SUBSPACE, NEAR-EXACT NUMBERS.

1 • Exact shape

$\tilde{W}$ is $l \times h$. Its width confirms Finding 1 a second time, independently of the singular-value count.

2 • Same subspace

$\text{span}(\tilde{W}) = \text{span}(W)$. The h -dimensional subspace of $\mathbb{R}^l$ is recovered exactly. G changes the basis, never the space itself.

3 • Near-exact numbers

After least-squares alignment the RMS error is $\sim 10^{-4}$. A random matrix scores $2 \cdot 10^{-2}$ — 100 to 500× worse.

We cannot know **which basis** the model uses. We do know, exactly, **which subspace** it projects into.

- That subspace is a genuine, permanent fact about a model whose weights have never been published.

PART 3

WHAT IT COST, WHAT CHANGED

The attack was run against live production models, with permission
— and the industry shipped fixes before the paper appeared.

Run against real production models

FIVE OPENAI MODELS, 1 JANUARY 2024.

Model	Hidden size	Queries (dim)	Cost (dim)
ada	1024 ✓	$< 2 \cdot 10^6$	\$1
babbage	2048 ✓	$< 4 \cdot 10^6$	\$2
babbage-002	1536 ✓	$< 4 \cdot 10^6$	\$2
gpt-3.5-turbo-instruct	withheld ✓	$< 4 \cdot 10^7$	\$200
gpt-3.5-turbo-1106	withheld ✓	$< 4 \cdot 10^7$	\$800

✓ exactly correct, confirmed privately by OpenAI · withheld at OpenAI's request · † not run, to preserve model security

- Quantisation is not a defence: at 8-bit and 4-bit the same h comes out in the same number of queries.

Disclosed before it was published

HOW THE WORK WAS ACTUALLY CONDUCTED.



- The recommended fix is **logit_bias XOR log-probs**. allow either, never both in one request. The attack is 10× more expensive without the combination.
- OpenAI specifically removed logit_bias's ability to affect the top log-probs.



Thank you

Questions?